

WEEK-4 TASK

Task Name:

Order Management System

Description

The Order Management System is developed to organize customer purchases and maintain detailed information about each order. The system uses Orders and Order_Details tables to record customers, products, quantities, prices, and order status. It allows users to add new orders, update existing order information, and retrieve customer-wise purchase details.

Objectives

1. Create and maintain Orders and Order_Details tables.
2. Record customer purchase information accurately.
3. Maintain product quantity and pricing details.
4. Update order records whenever changes are required.
5. Generate reports showing customer purchase history.

Tables

1. Orders

- order_id – Unique identifier for each order
- customer_id – Identifies the customer
- order_date – Date on which the order was placed
- total_amount – Complete amount of the order
- order_status – Current status of the order

2. Order_Details

- order_detail_id – Unique identifier for each order item
- order_id – References the related order
- product_id – Identifies the purchased product
- quantity – Number of units purchased
- unit_price – Price of one unit
- subtotal – Total price for the selected quantity

SQL Implementation

```
CREATE DATABASE OrderManagementDB;
```

```
USE OrderManagementDB;
```

```
CREATE TABLE Orders (  
    order_id INT PRIMARY KEY,  
    customer_id INT,  
    order_date DATE,  
    total_amount DECIMAL(10,2),  
    order_status VARCHAR(20)  
);
```

```
CREATE TABLE Order_Details (  
    order_detail_id INT PRIMARY KEY,  
    order_id INT,  
    product_id INT,  
    quantity INT,  
    unit_price DECIMAL(10,2),  
    subtotal DECIMAL(10,2),  
    FOREIGN KEY (order_id) REFERENCES Orders(order_id)  
);
```

Insert Operations

```
INSERT INTO Orders VALUES  
(201, 10, '2026-08-17', 3200.00, 'Confirmed'),  
(202, 11, '2026-08-17', 1850.00, 'Pending'),  
(203, 12, '2026-08-18', 4500.00, 'Shipped');
```

```
INSERT INTO Order_Details VALUES  
(1, 201, 301, 2, 1200.00, 2400.00),
```

```
(2, 201, 302, 1, 800.00, 800.00),  
(3, 202, 303, 1, 1850.00, 1850.00);
```

Modification Operations

```
UPDATE Orders
```

```
SET order_status = 'Shipped'
```

```
WHERE order_id = 201;
```

```
UPDATE Order_Details
```

```
SET quantity = 3,
```

```
    subtotal = 3600.00
```

```
WHERE order_detail_id = 1;
```

Customer Order History

```
SELECT
```

```
    o.order_id,
```

```
    o.customer_id,
```

```
    o.order_date,
```

```
    o.order_status,
```

```
    od.product_id,
```

```
    od.quantity,
```

```
    od.unit_price,
```

```
    od.subtotal
```

```
FROM Orders o
```

```
JOIN Order_Details od
```

```
ON o.order_id = od.order_id
```

```
WHERE o.customer_id = 10;
```

Expected Output

Order ID	Customer ID	Order Date	Status	Product ID	Quantity	Unit Price	Subtotal
201	10	2026-08-17	Shipped	301	3	1200.00	3600.00
201	10	2026-08-17	Shipped	302	1	800.00	800.00

Conclusion

The Order Management System provides an efficient database structure for handling customer purchases. By connecting the Orders and Order_Details tables, the system can maintain order information, product quantities, pricing, and order status. SQL operations such as insertion, modification, and customer-wise reporting make the system useful for managing order records effectively.